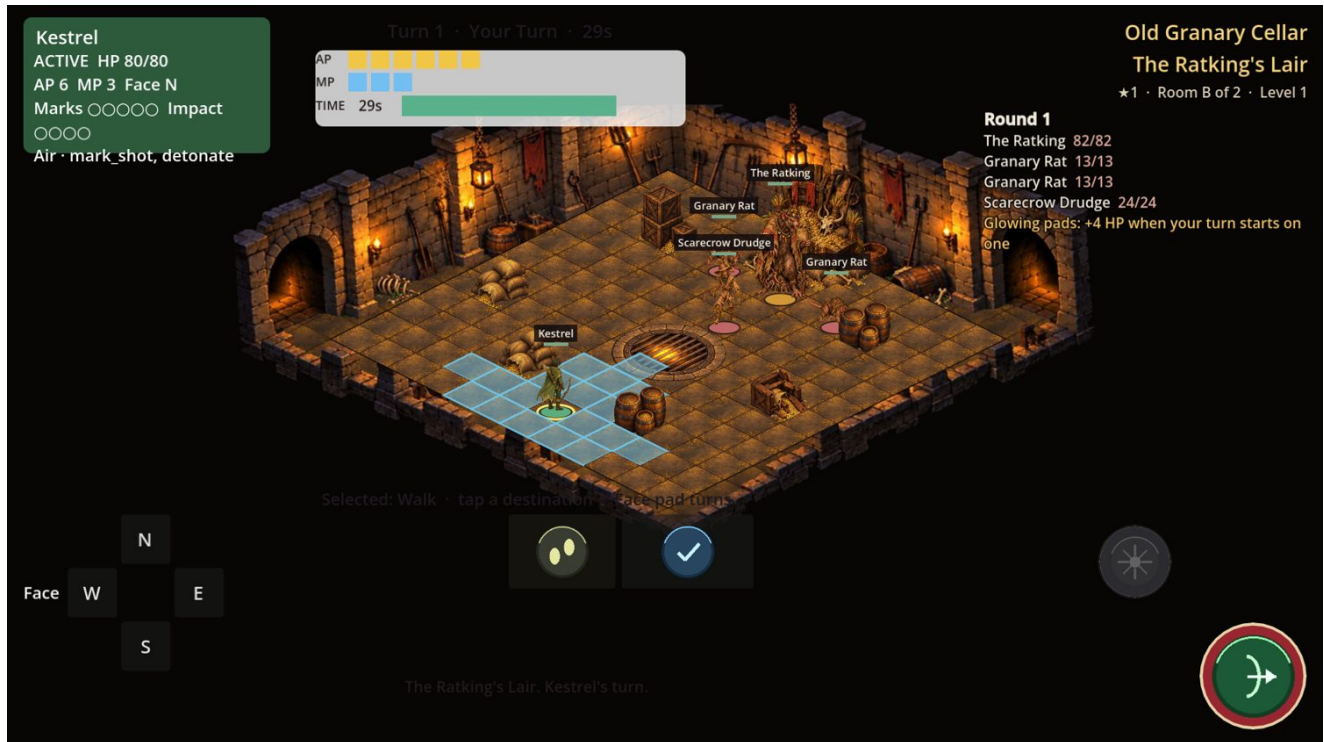


How we built the dungeon rooms

A step-by-step guide for the mobile team and its agents, from the PC build of dungeon 1, the **Old Granary Cellar** (Stoneford, levels 1–10).



In game on PC: room B, the Ratking and his escort on the painted lair board (★1).

Who this is for

Agents on the mobile team who will build something similar for the phone version. Every step below is what we actually did, with the real file names, numbers and prompts, so you can repeat it. Where the phone needs something different, it says so (section 12).

The source of truth

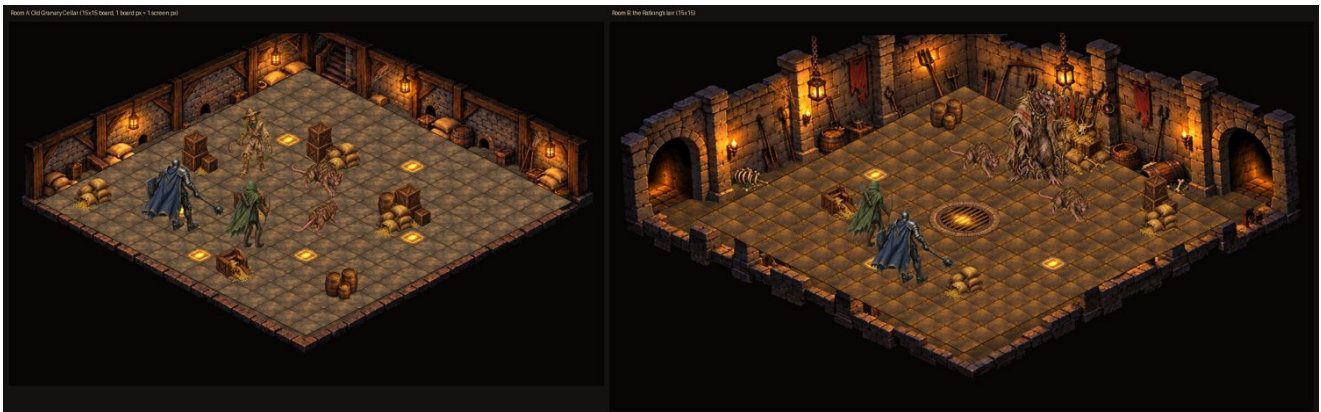
Everything here lives on branch **pc/world-zones** of **maurogp12/stasiumxii** (merged as PR #293). Read the files themselves before you copy a number; this PDF explains them, it does not replace them.

What	Where (branch pc/world-zones)
Art kit + its README (the best single read)	art/pc/dungeons/old_granary_cellar/README.md, manifest.json
Art build scripts and the chosen source paintings	build_tools/dungeons/ (gkit.py, build_granary_board.py, mrig.py, granary_monsters.py, granary_src/)
Room layouts, run file, design choices	data/world/dungeons/old_granary_cellar/ (README.md, build_rooms.py, room_a_tags.json, room_b_tags.json, run.json)
Monster stats and star rows	data/world/dungeon_monsters.json, data/world/dungeons.json
Rules code	backend/pc_dungeon_run.gd, pc_monsters.gd, dungeon_ai.gd, world_dungeons.gd, dungeon_section of combat_sim.gd
Drawing code	scenes/world/dungeon/ (dungeon_board_view.gd, dungeon_art.gd, monster_pawn.gd, dungeon_run.gd ...)
Tests and balance sim	tests/run_dungeon_tests.gd (684 checks), tests/sim_granary_stars.gd
The approved designs and the plan	branch claudes/pc-zones-spec: docs/pc/look_target/dungeons/ (DUNGEON_PLAN.md + images)

1. The whole process on one page

A dungeon is two rooms on the normal turn-based combat board: **room A** is a pack fight, **room B** is the boss with an escort. The look comes from paintings, but the game never uses a painting as one flat picture: every piece (floor, props, walls, each monster) is painted on its own and cut out, so the grid, clicks and depth sorting still work.

1. **Design** – paint 4 look-target images per dungeon (entrance, room A, boss room, monster sheet). Mauro approves them.
2. **Paint the pieces** – repaint each piece alone on a flat key colour, using the approved images as references.
3. **Cut them out** – chroma-key to binary alpha, project floors onto the iso diamond, fit the room shell onto the board.
4. **Lay out the rooms** – a small Python script writes the board tags JSON (props, pads) for each room.
5. **Animate the monsters** – cut each monster painting into parts and pose them frame by frame (painted-part rig).
6. **Write the manifest** – one JSON lists every PNG with size, pivot, footprint and frame counts. The game loads only through it.
7. **Rules** – monsters, AI, line of sight, boss summon, escort, pads and the ★5 hazard go in CombatSim.
8. **Stars and rewards** – ★1–★5 per run: tougher monsters, bigger packs, more XP, coins and better loot.
9. **Prove it** – tests, a balance sim, stills and a gameplay video. Send them to Mauro and wait for his OK.



Room A (left) and room B (right) assembled from the cut-out pieces, before any code: the review mock made by `mock_granary_rooms.py`.

Mauro's rules that apply to every dungeon

- One dungeon at a time. When one is done, send Mauro in-game stills, a gameplay video and the monsters, then **ask before starting the next**.
- The boss always fights with monsters beside him (an escort), plus any summons.
- Stars work like mobile: higher star = tougher monsters, more XP, better loot, more coins. ★5 brings a mutated boss form (for the Granary: a radioactive Ratking).
- At least 6 monsters in the first room, including ranged ones (the Sling Rats).

2. Step 1 – The design images

We used **Scenario** (team “Maurogarza06's Organization”, project **stasium**), model **gpt-image-2 (sunburst)**, with a real game screenshot uploaded as the style reference so the painting matches the game's camera and look. Four images per dungeon:

Image	Shows
1_door_keeper.jpg	The entrance in town with the Door Keeper beside it
2_room.jpg	Room A, the pack fight, on the glowing themed floor
3_boss_room.jpg	Room B, the boss, his escort and the throne
4_monsters.jpg	The monster sheet (every monster of the dungeon)

These are **look targets**, not game art. Once Mauro approves them they become the reference image for every piece painted in step 2, which is what keeps all the pieces in one style. All five town dungeons have approved designs; the asset ids are in DUNGEON_PLAN.md.

How to get images out of Scenario here

- Upload a reference: *upload_asset* (multipart) → PUT the file to the returned URL → *upload_asset_complete*.
- Generate: *model_run*, then *jobs_wait*.
- Our cloud machine could not reach the Scenario CDN, so we saved each result by calling *asset_display* and copying the JPG it writes into the session's tool-results folder. If the CDN is allowed in your environment, *asset_download* is simpler.
- Keep the chosen file in the repo (we keep them in `build_tools/dungeons/granary_src/`) and write its asset id in the README, so anyone can trace it back.

3. Step 2 – Paint every piece on its own

Each piece is a separate Scenario image, with the approved design (and, for monsters, the monster sheet) attached as reference. Four rules made the cut-outs clean:

- **One piece per image**, large and centred with margin around it.
- **Flat key background**: pure green #00FF00. Use **magenta #FF00FF** for anything that is itself green (the ★5 toxic pieces).
- **No floor, no ground plane, no cast shadow, no other objects** in prop and monster images. The game draws its own shadows.
- **Say the camera exactly**: “the same isometric 2:1 game camera as the reference” for props and walls; “DIRECTLY ABOVE (orthographic top-down)” for floor tiles and puddles.



Left: a floor flagstone, painted top-down edge to edge with a grout line on all four sides. Right: a prop on flat green.

Prompts we used (copy the pattern, change the subject)

Floor tile (3 variants per room):

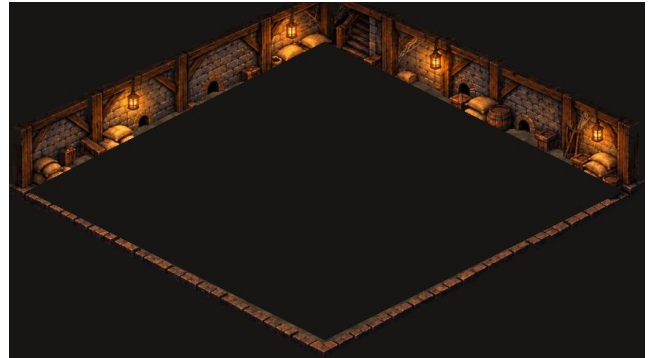
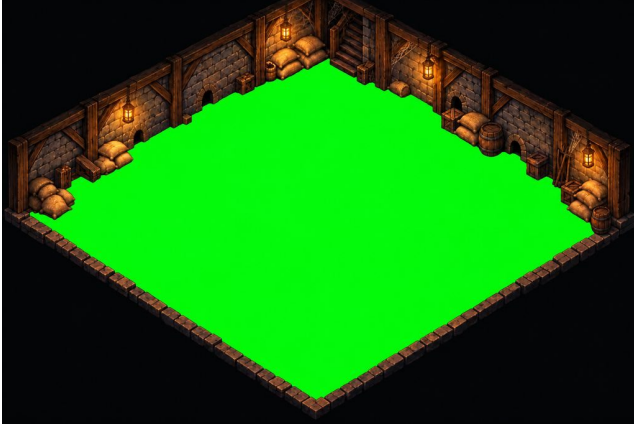
A single square stone floor flagstone from the cellar floor in the reference image, seen from DIRECTLY ABOVE (orthographic top-down, no perspective), filling the entire image edge to edge. One worn grey-brown cellar flagstone with soft hand-painted texture, small chips and hairline cracks, a few scattered golden wheat grains, lit evenly by warm lantern light, the same painted colours as the reference floor. A thin dark grout line runs along all four edges of the image. No objects, no border decoration, no shadows from objects, no text.

Prop:

One isolated game prop from the cellar in the reference image: a CRATE STACK, two weathered wooden crates with X braces stacked plus one smaller crate beside them, ... Same hand-painted style, colours and warm lantern lighting as the reference. Seen from the same isometric 2:1 game camera as the reference (one box corner pointing at the viewer). Large and centred in frame with margin around it. No floor, no ground plane, no cast shadow, no other objects. Background: flat solid pure green #00FF00, one uniform colour.

Room shell (the walls; one per room). The trick: the floor diamond is painted as one flat green area so we can find it and fit it onto the board exactly.

Repaint the cellar room from the reference image as an EMPTY ROOM SHELL for a game backdrop, same isometric 2:1 camera, same hand-painted style and warm lantern light. Keep exactly: the two back walls ... hanging iron lanterns ..., small arched rat holes at the wall base, cobwebs, the wooden stair in the back corner ...; and the two front edges as a low rough stone block lip one block high. Remove EVERYTHING on the floor. The whole floor diamond inside the walls and lip must be ONE flat solid pure green #00FF00 colour with no texture, no tiles, no shadows, so it can be cut out. Everything outside the room is plain near-black dark. Show the whole room, all four corners.



Left: the shell as painted. Right: the game backdrop after the fit (step 3). The board is drawn inside the cut-out floor.

Monster (two paintings per monster: S = three-quarter front facing lower-right, E = three-quarter back facing upper-right):

THE RATKING boss from the reference monster sheet (right creature), painted again as a single game character, identical design: ... View: three-quarter FRONT view from a slightly elevated isometric game camera, body and face turned toward the LOWER-RIGHT of the image. Neutral standing pose ..., the free clawed hand held a little away from the body. Full figure crown to feet inside the frame with margin, large in frame. No ground, no shadow. Background: flat solid pure green #00FF00.

Ask for **limbs held a little away from the body**. The rig in step 5 cuts arms, legs, tail and weapon out as separate parts, and that only works if they don't overlap the torso.

★5 mutated form: use an *edit* of the base painting ("repaint this exact same Ratking, in exactly the same pose, framing, size and silhouette, as his RADIOACTIVE mutated form ..."). Because the silhouette stays the same (mask overlap 0.92–0.95), the ★5 version reuses the base rig's part cuts and costs almost nothing extra.

4. Step 3 – Cut the pieces out (build_tools/dungeons/)

All cutting is Python (numpy, OpenCV, Pillow). **gkit.py** holds the shared helpers; the builders call it.

Chroma key (gkit.key_green)

- Alpha comes from the **green excess** = $G - \max(R, B)$: fully opaque at excess ≤ 18 , fully clear at ≥ 200 .
- The colour is **un-mixed** from the known key green, and green is clamped to $\max(R, B) + 12$ on any partly keyed pixel (despill), so no green fringe survives.
- Downsample in premultiplied space (area filter), then cut alpha to **binary 0/255** and set RGB to 0 under alpha 0. Every game PNG is binary alpha, the same as the hero sheets.
- Write a 1x file and a **_2x/** master. The game draws the 2x at scale 0.5 in the same place.

Floor tiles onto the iso diamond

- The board math is the Koliseo's **board_view.gd**: $\text{cell_to_local}(x, y) = ((x - y) \cdot 32, (x + y) \cdot 16)$, the diamond centre. Tiles are 64x32 (128x64 for 2x).
- Crop the painted slab on its own grout line (grout_crop), then map the square onto the diamond: top tip = texture corner (0,0), +x runs to the right tip. Neighbouring tiles then share one grout line and the floor tiles seamlessly. Add a 0.5 px bleed on the diagonal edges.
- Pick the variant per cell with the kit hash $h(x, y, 3)$, so a room never looks like a checkerboard.

Props

- Scale each prop to a target width for its footprint (crate stack 66x82 px at 1x for 1x1; the bone throne 152x156 px for 2x2).
- Anchor: the image's **bottom-centre sits on the south tip of its south-most cell**. Props y-sort with the units.
- We graded the grain props down a little ($\times 0.9$, saturation 0.9) to sit in the darker cellar.

Room shell → backdrop (build_granary_board.backdrop)

1. Find the biggest connected area of strong green (excess > 120): that is the painted floor.
2. Find its four corners and solve a **least-squares affine** that maps them onto the board diamond corners (0,-16), (32n, 16n-16), (0, 32n-16), (-32n, 16n-16) for an n×n board.
3. Warp the whole painting with it, key the near-black surround to alpha 0 and cut the floor out.
4. Paint the floor outside the board with the same tiles, darkened, up to the walls, so no gap shows.
5. Write one backdrop per room per board size (we made 12x12 and 15x15) and record the pixel where cell (0,0) lands (cell00_centre_px).

Room B's painted floor was not an exact parallelogram, so a 15 px error stayed at the corners. The darkened floor apron hides it.

5. Step 4 – Lay out the rooms

Each room is a board in the **same tags JSON format as the Koliseo maps** (and mobile's art/maps/stasis_v1/*_tags.json): size and a list of cells with x, y, terrain, elevation, paint_only. Dungeons add two optional keys: blocks (a prop stands here: not walkable, blocks a body) and special: "pad" (a glowing pad). Don't hand-edit the JSON: a short script writes it.

```
# data/world/dungeons/old_granary_cellar/build_rooms.py (shortened)
ROOM_A = {
    "props": {(1,6): "grain_sacks", (9,1): "barrel_cluster", (5,5): "crate_stack", ...},
    "pads": {(4,3): "wheat_pad", (8,4): "wheat_pad", (3,6): "wheat_pad", (7,7): "wheat_pad"},
}
ROOM_B = {
    "props": {(5,0): "bone_throne:part", (6,0): "bone_throne:part", (5,1): "bone_throne:part",
              (6,1): "bone_throne", (2,2): "crate_stack", ...}, # a 2x2 prop names itself on its
    "pads": {(x,y): "drain_grate" for x in range(5,8) for y in range(5,8)}, # south (anchor) cell
}
# each cell -> {x, y, terrain:'ground', elevation:0, paint_only:[name], blocks?:true, special?:'pad'}
```

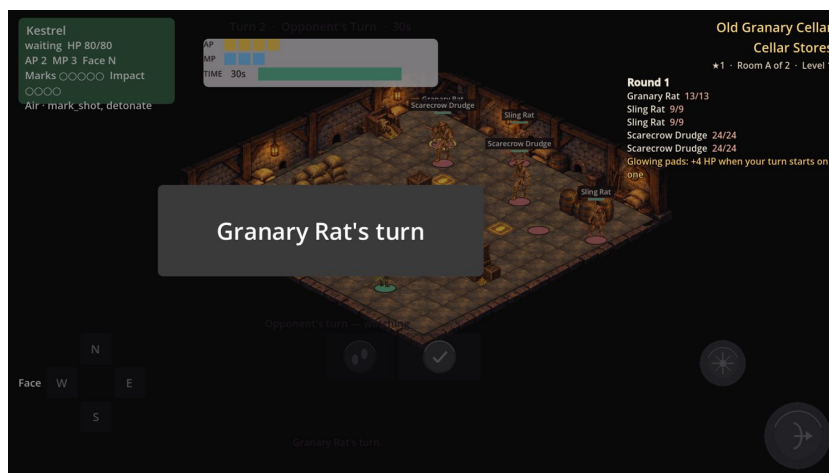
Layout rules we followed

- 12x12 boards (the backdrops also exist for 15x15).
- Props break up lines of fire and make cover for the ranged monsters, but **never seal a route**: every walkable cell must stay reachable.
- Monsters spawn on the far rows, off the pads, away from the hero's start (hero at (8,10) facing N).
- Room B: the boss in front of his throne against the back wall, the escort in front of him, the glowing 3x3 drain grate in the middle.
- Pads heal the hero 4 HP when a turn starts on one. Monsters get nothing from them. Full heal on the stairs between rooms.

The run file (run.json)

One file describes the whole run: board size, the art manifest path, the pad rule, the between-rooms heal, and per room the tags file, hero start and the monsters. A room has packs keyed by star: the game uses the pack for the highest key at or below the chosen star.

Room A pack	Granary Rats	Sling Rats (ranged)	Scarecrow Drudges	Total
★1–★2	2	2	2	6
★3–★4	3	2	2	7
★5	3 radioactive	3 radioactive	2	8



Room A in game (★1): 6 monsters, wheat pads, props, the painted cellar shell. This shot also shows our biggest mistake: the board is too small on a big screen (see section 13).

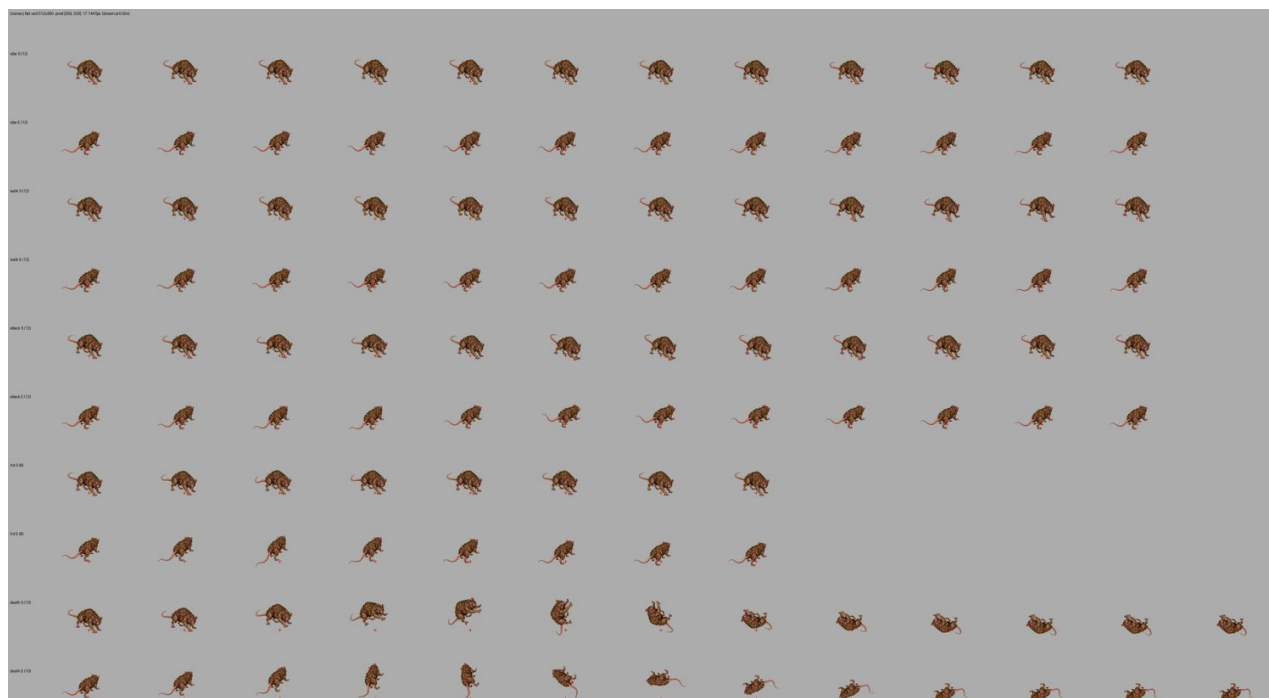
6. Step 5 – Animate the monsters (painted-part rig)

The same idea as the hero action rigs, done in 2D straight from the two paintings with no 3D blockout (**mrig.py** + **granary_monsters.py**):

1. **Cut parts** (head, limbs, tail, weapon, feet) out of the painting with polygons. Each part pivots on a painted joint.
2. **Back-fill** what a part covered on the body with the painting's own texture (shifted patches). Fill holes enclosed by the body too.
3. **Pose each frame** by forward kinematics, render at 3x on the cell, area-downsample, cut to binary alpha, fill see-through specks.
4. **Check every frame lies inside the cell.** If a pose would leave it (only the deaths did), a smoothed lift eases it back in instead of popping.

Monster	Cell / pivot	Actions (frames)
granary_rat, sling_rat, scarecrow_drudge	512x360 / (256,329), the hero cell	idle 12, walk 12, attack 12, hit 8, death 13
the_ratking (boss)	768x540 / (384,494), 1.5x	the same + summon 14

- 17.144 fps. Idle and walk loop; the others play once and death holds its last frame.
- Paint only **S** (front, screen down-right) and **E** (back, screen up-right); W and N are flip_h mirrors in the game.
- Ranged attack: the **release frame** (Sling Rat f07) and the sling-pocket position at release per facing are stored in meta.json (release.point_px), so the stone leaves the hand exactly. Mirror x for W and N.
- Boss signature (summon): crook raised and trembling f03–f09, slammed down f10; the summoned rats spawn around f09.
- ★5 frames each get a same-size additive <frame>_glow.png (the green paint bloomed), drawn on top with an add blend.



Granary Rat contact sheet: every frame of every action, S and E. Make one of these for every monster and look at it before you ship.



★5: the base Ratking (left) and the Radioactive Ratking S and E. Painted as edits of the base, so they reuse its rig.

Known weak spots of this rig (fix them on mobile if you can)

- **Deaths have no painted lying-down pose:** the standing painting is rotated (the rat rolls 180°, the Ratking ~84°), so you can see the cut where his feet were. Paint a lying key per monster and facing.
- Weapon swings rotate a rigid painted piece about the shoulder; the sickle points backward in the wind-up.
- Back-fill repeats nearby texture (a repeated wheat ear behind the rat's hind leg).

7. Step 6 – The manifest and the loader

manifest.json lists every PNG (1,276 for the Granary) with its kind, size, pivot or anchor, footprint or cell, frame counts and fps. **build_granary_manifest.py** writes it and fails if any PNG is missing from it. The game reads art **only** through it (**dungeon_art.gd**):

- Lookups: building (town door), floor, backdrop, props and pads per room, monster strips per action and facing.
- The loader reads keys loosely, so new painted files drop in without code changes.
- If a file or the manifest is missing, a clearly drawn **placeholder** stands in. That let the code team build and test the whole run before the art landed, and swap the art in later with no code change. Do the same: art and code can then run in parallel.

Drawing order on the board (**dungeon_board_view.gd**)

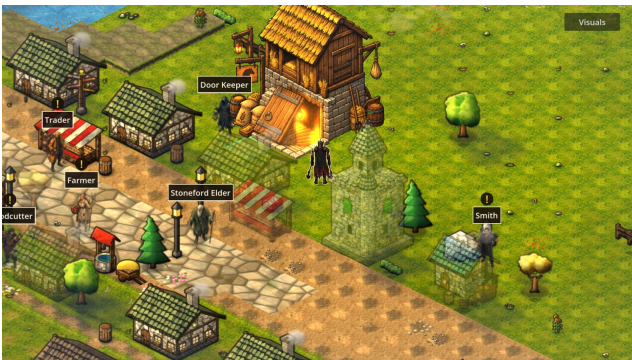
1. Clear to near-black, then draw the backdrop so `cell00_centre_px` lands on `cell_to_local(0,0)`.
2. Floor tiles centred on each cell; pad tiles replace the floor on pad cells; the additive pad glow on top (optionally pulsing).
3. Ground decals (drain grate, ★5 toxic pools).
4. Props and units y-sorted together (each prop by its south cell).
5. Fit the camera to the backdrop's bounds so the back walls show.

Monster pawns (**monster_pawn.gd**)

The existing Pawn keeps doing the motion work (idle bob, step bounce, lunge, recoil, slump) on an invisible stand-in; the monster pawn copies that motion onto its own body, which plays the frame strips. So monsters get all the board's existing feel for free. The boss is placed so his (384,494) pivot lands where a hero's (256,329) would. We also tinted the flat, cool frames for the cellar lantern light (lift red, cut blue).

Town door

The entrance is one painted 3x3 building sprite in the town kit style, plus an additive hatch glow that fades in on hover. The door cell is passable, in front of the stairs; the existing Door Keeper NPC stands beside it. Clicking either opens the entry panel (name, level band, party size, star picker, Enter).



The granary door and the Door Keeper in Stoneford; the entry panel with the star picker.

8. Step 7 – The rules (CombatSim)

Dungeon fights are the **same CombatSim** as the Koliseo, with a dungeon section added (one hero seat, several monster seats). Mobile already has its own combat_sim.gd and Stasis rooms, so you will be adding to those, not starting over.

Monster (level 1)	HP	AP	MP	Attack
Granary Rat	13	4	3	Gnaw 4, range 1
Sling Rat	9	4	3	Sling 3, range 2–5, needs line of sight
Scarecrow Drudge	24	5	2	Sickle 6, range 1
The Ratking	82	6	2	Crook 10, range 1; Call the Swarm

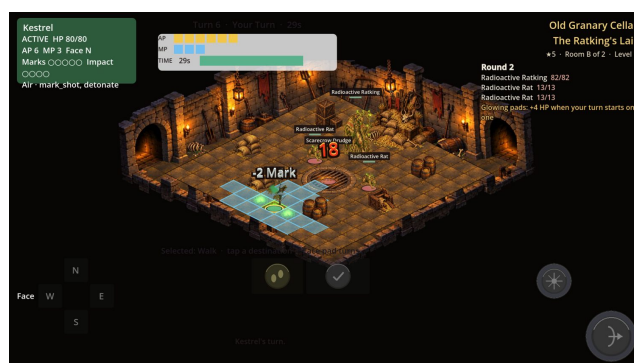
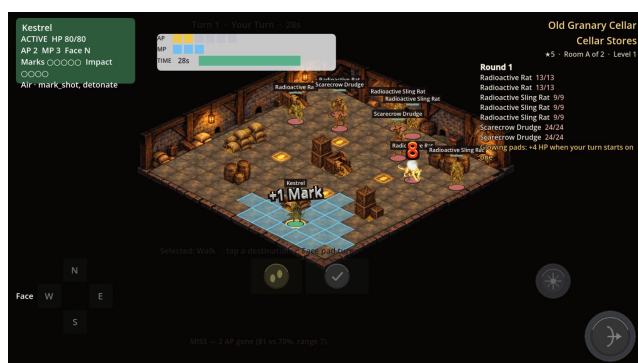
+5% HP and +3% damage per level above the band's first level. Stats live in data/world/dungeon_monsters.json.

Monster AI (dungeon_ai.gd), one intent per call

1. Boss signature (summon, pools) when CombatSim says it is ready.
2. Attack the hero if in range and AP covers it.
3. Otherwise walk: flood-fill (breadth-first) over walkable cells and step toward the nearest cell that can attack. Bodies only block the final stand cell, so a monster stuck behind another takes the next lane.
4. Face the hero once, then end the turn. Every call spends AP or MP, turns, or ends the turn, so a turn always ends (hard cap 12 steps).

What we added to the rules

- **Line of sight** for ranged monsters (has_los): the straight line may not cross a prop cell; bodies do not block. Sling Rats kite: next to the hero they step back to range 2+ first.
- **Escort**: room B always has the boss plus 2 rats and 1 Drudge. The room is won when every monster is down.
- **Summon with one shared cap**: the Ratking calls 2 rats on his 2nd turn and every 3rd turn after. The cap counts every live rat of that kind, escort included: 4 alive, 4 summoned in all (★5: 6). When he falls, his summons scatter; the escort fights on.
- **Pads**: +4 HP for the hero at turn start on a pad.
- **★5 toxic pools**: every other boss turn, 2–3 cells around the hero (the hero's cell first) become pools for 3 hero turns. Ending a turn on one costs 6 poison; walking through is safe. Danger cells glow green with their turns left. Radioactive Sling Rats add 2 poison for 2 turns (no stacking).



★5: the radioactive pack in room A; the Radioactive Ratking and his toxic pools in room B.

9. Step 8 – Stars and rewards

The player picks ★1–★5 per run on the entry panel (no unlock gate, as on mobile). The best star cleared is saved and shown. We copied mobile's model from stasis_catalog.gd, hero_progress.gd and gear_bag.gd.

Star	Monster scale [HP, dmg] (PC solo)	Room A size	Loot	Sim win %
★1	[1.0, 1.0]	6	Normal parts	85%
★2	[1.1, 1.05]	6	Normal parts	68%
★3	[1.15, 1.1]	7	Rare parts unlocked	60%
★4	[1.2, 1.1]	7	Rare parts	58%
★5	[1.0, 1.0] + radioactive pack, boss and pools	8	Guaranteed Rare + Mystery Box	38%

- **XP** = the base win XP × star (mobile: 60 × star). **Coins** = the dungeon's coin roll × star.
- **Why PC's rows are smaller than mobile's:** mobile's STAR_SCALE {1:[1.0,1.0], 2:[1.4,1.2], 3:[4.0,2.8], 4:[6.8,4.2], 5:[8.8,5.2]} is tuned for a **party of 4** from ★3 (PARTY_FOR_STAR, PARTY_SCALE). PC runs are solo, and the raw rows gave 0% wins at ★3+. On mobile, **keep your own party tables** and tune with your sim (tests/sim_dungeons.gd).
- ★5's difficulty comes from new mechanics (mutated boss, radioactive pack, pools), not just bigger numbers. That reads better to players than a stat wall.
- Tuning loop: run the sim per class per star, change one row, run again. ★4 at [1.25, 1.15] gave 30%, so it was eased to [1.2, 1.1].



★5 win screen: XP and coins × star, the guaranteed Rare and the Mystery Box.

10. Step 9 – Prove it, then ask Mauro

- **Tests:** tests/run_dungeon_tests.gd (684 checks) covers data loading, room layouts (every cell reachable, props never sealing a route), pack sizes per star, the AI always ending its turn, LoS, the summon cap, pools, rewards per star, and that a saved best-star from disk cannot leak into a test.
- Run **every** headless suite before any merge, one at a time with its own save folder (XDG_DATA_HOME), and write each result to its own file.
- **Balance sim:** tests/sim_granary_stars.gd plays the hero with a bot, 8 runs × 5 classes per star, and prints win rates.
- **Captures:** dungeon_run.tscn takes `-- --dungeon-capture stills|showcase|run [--class kestrel]` and records stills, a monster showcase and a full run (xvfb + --rendering-driver opengl3 on a headless machine).
- **Send Mauro** the stills (door, room A, room B), a full-run video and the monster showcase, with honest notes on what is weak. **Wait for his OK** before the next dungeon.

11. Our numbers, for scale

Item	Granary
Scenario images chosen	about 35 (floors, props, shells, 2 per monster, ★5 edits, projectiles)
Game PNGs in the manifest	1,276
Monsters	4 + 3 ★5 versions
Rule and view code	about 4,000 lines across 13 files, plus about 740 lines of dungeon rules in combat_sim.gd
Tests	684 checks + a balance sim

12. Doing the same on the phone

Mobile already has the skeleton: Stasis dungeons with 2 rooms (trash pack → boss), ★1–★5, parties, foes and bosses (backend/stasis_catalog.gd, stasis_ai.gd, foe_kits.gd, scenes/stasis_run.gd, stasis_fight.gd, 15x15 room tags in art/maps/stasis_v1/, boss sheets in art/stasis/bosses/). So the job is mostly to **upgrade the look and add the new mechanics**, not to build a new mode. Check each mapping below in the mobile code before you rely on it.

PC piece	Likely mobile home	Notes
Look targets (4 images per dungeon)	Same process, new images per Stasis biome	Use a phone screenshot as the style reference so the camera and scale match the phone.
Room tags + build_rooms.py	art/maps/stasis_v1/*_room_{a,b}_15x15_tags.json	Same format. Add blocks / special: pad only if the mobile loader reads them.
Backdrop + floor kit + props	board/ drawing (tile.gd, koliseo_art.gd)	Our backdrops already exist in 15x15. Ship 1x only on phones (memory).
Painted monster rig	art/stasis/foes, art/stasis/bosses	Mobile sheets are per action × 4 facings. Our S/E + mirror halves the art.
dungeon_ai.gd	stasis_ai.gd	Port the flood-fill walk, kiting and LoS if mobile foes lack them.
Summon cap, escort, pools, pads	combat_sim.gd (mobile copy)	Keep the rule text in data so tests can read it.
Star rows	stasis_catalog.gd STAR_SCALE / PARTY_SCALE	Keep mobile's party tuning; don't copy PC's solo rows.
Tests + sim	tests/ + tests/sim_dungeons.gd	Add room reachability, AI-always-ends-turn and summon-cap checks.

Phone-specific advice

- **Make the board big and the targets fat from day one.** On PC the board came out too small and Mauro had “lots of trouble selecting monsters”. On a phone it is worse: let a tap on a monster's **sprite** pick its cell, highlight cells in range, and never let tooltips cover the board.
- Pause or extend the turn timer while the player is aiming.
- Ship 1x textures only and load monster frames per room, not all at once. Phones have far less memory: on PC, switching to 1x textures (with the animations held still) roughly halved texture memory.
- Keep the per-run star picker and the gate rule: one dungeon at a time, stills + video + monsters to Mauro, then ask.

13. What went wrong, so you don't repeat it

Problem	What we did / what to do
Board too small on big screens; monsters hard to click	Fit the camera to the room, click the sprite not just the cell, show in-range cells. (A PC fix was in progress when this guide was written.)
Deaths look odd (rotated standing painting)	Paint a lying-down key per monster and facing.
Scenario CDN blocked in the cloud machine	Save results via asset_display, or ask Mauro to allow the CDN host in the environment's network settings.
Tests failing on a fresh checkout	Run a Godot --import first; the import cache was stale.
A best-star save on disk leaked into tests	Give every test run its own save folder and reset saved state in the test.
Mobile STAR_SCALE impossible for one hero	Tune per party size with a sim; never copy rows across modes without re-running it.
Disk filled up mid-run	Delete old worktrees and build outputs; write suite results to small files.
Agents stopped by restarts	Push work early and often; keep results in named files so work can resume.

14. Checklist for each new dungeon

- [] 4 look-target images approved by Mauro
- [] Every piece painted alone on flat green (or magenta), chosen files and asset ids saved
- [] Floors, props, glows, shells cut out; 1x (+2x on PC) binary alpha; backdrops fitted
- [] Room tags written by a script; every cell reachable; packs per star in the run file
- [] Each monster: S + E paintings → rig → contact sheet reviewed; boss signature; ★5 form + glow maps
- [] Manifest written and checked; the game loads only through it; placeholders work
- [] Rules: AI, LoS for ranged, escort, summon cap, pads, ★5 hazard
- [] Stars: scale rows tuned with the sim; XP, coins, loot gates
- [] All suites green; stills + full-run video + monster showcase sent to Mauro; his OK before the next one